Points & Rewards Service: Entity Reference

Every entity to build, grouped by module, with its purpose, fields and milestone.

Build order by milestone

Build only what each milestone needs. The base classes come first because every entity extends one of them.

Milestone	Entities to create	Why they're needed at this point
M1. Core points engine	BaseEntity, AuditableEntity, SoftDeletableEntity, User, PointAccount, LedgerEntry	Earn, redeem, balance and history, plus the concurrency test
M2. Security	UserCredential, UserRole, ExternalIdentity, RefreshToken, AuditLog, IdempotencyRecord	Password and Google login, JWT refresh, roles, audit trail, safe retries
M3. Campaigns and patterns	Campaign, PointLot	Campaign rules (Strategy pattern), oldest-first redemption, 12-month expiry
M4. Kubernetes, 3 replicas	No new entity; add the ShedLock table via Flyway	Stops the expiry job running on all 3 replicas at once
M5. Events and resilience	OutboxEvent	Transactional outbox so no event is lost if the app crashes
M6. Proof and polish	None	Dashboards, load test and README only

Rules that apply to every entity

- **IDs:** UUIDv7 generated by Hibernate. Time-ordered, so indexes stay efficient, and safe to expose in APIs.
- **Between modules, store only IDs.** PointAccount holds `userId`, never a JPA relationship to User. Modules stay independent.
- **No public setters.** Create through a static factory (for example `User.create()`); change through named methods (`suspend()`, `deduct()`). Use Lombok `@Getter` only, never `@Setter` or `@Data`.
- **Constructors:** `@NoArgsConstructor(access = AccessLevel.PROTECTED)` for JPA and Hibernate proxies.
- **Types:** primitives (`long`, `boolean`) for required fields; wrappers only where null has a meaning. Points are always `long`, never `double`.
- **Enums:** stored with `@Enumerated(EnumType.STRING)`, and placed in the module's `domain/` folder.
- **Schema:** every table is created by a Flyway migration, never by Hibernate auto-DDL.

shared/persistence: base classes (M1)

Three `@MappedSuperclass` layers. Each entity extends the most restrictive one that fits, so the class itself tells you whether the data can change.

BaseEntity

Extends: nothing | **Milestone:** M1 | **Table:** `(no table of its own)`

What it's for: Fields every entity has. Entities that extend only this one are created once and never changed (ledger, audit log).

Field	Type	Rules and meaning
`id`	UUID	Primary key, UUIDv7, set by Hibernate on save. Never updatable.
`createdAt`	Instant	Set automatically by JPA auditing. Not null, never updatable.
`createdBy`	UUID	User who created it, from AuditorAware; the system ID for background jobs.

Also holds the final equals() and hashCode(): equal when the ID is the same and non-null; hashCode is constant per class.

AuditableEntity

Extends: BaseEntity | **Milestone:** M1 | **Table:** `(no table of its own)`

What it's for: For entities that can be updated. Adds who changed it last, when, and optimistic locking.

Field	Type	Rules and meaning
`updatedAt`	Instant	Set automatically on every update. Not null.
`updatedBy`	UUID	Who made the last change.
`version`	Long	@Version for optimistic locking. Wrapper type on purpose: null tells Spring Data the entity is new.

SoftDeletableEntity

Extends: AuditableEntity | **Milestone:** M1 | **Table:** `(no table of its own)`

What it's for: For business data that should be hidden rather than removed (users, campaigns).

Field	Type	Rules and meaning
`deletedAt`	Instant	Null means active. Set by markDeleted(clock); cleared by restore().

Each concrete entity also declares @SQLDelete and @SQLRestriction("deleted_at IS NULL") with its own table name. Unique columns need a partial unique index WHERE deleted_at IS NULL.

user module

User

Extends: SoftDeletableEntity | **Milestone:** M1 | **Table:** `users`

What it's for: The person's identity in the system: who they are, not how they log in. Every other module refers to a user only by its ID.

Field	Type	Rules and meaning
`email`	String (320)	Not null. Stored trimmed and lowercase. Unique among non-deleted users (partial index).
`displayName`	String (100)	Optional name shown in the app.
`status`	UserStatus	Not null. ACTIVE on creation.

Methods: create(email, displayName), changeDisplayName(), suspend(), activate(), isActive(). No password here; that lives in auth.

`UserStatus` (enum, used by User)

`ACTIVE, SUSPENDED`

points module

PointAccount

Extends: AuditableEntity | **Milestone:** M1 | **Table:** `point_accounts`

What it's for: One per user. Holds the current balance so balance reads are fast, and is the row that gets locked when two redeems race each other.

Field	Type	Rules and meaning
`userId`	UUID	Not null, unique (one account per user), never updatable.
`balance`	long	Not null, starts at 0. Database CHECK balance >= 0. Must always equal the sum of ledger entries.

Methods: openAccount(userId), add(amount), deduct(amount). deduct() throws InsufficientPointsException. The inherited version field gives optimistic locking for the concurrency test.

LedgerEntry

Extends: BaseEntity + @Immutable | **Milestone:** M1 | **Table:** `ledger_entries`

What it's for: Append-only history of every points movement. The source of truth: balances can be rebuilt from it, and mistakes are fixed with new REVERSAL entries, never edits.

Field	Type	Rules and meaning
`accountId`	UUID	Not null. The PointAccount this entry belongs to.
`type`	LedgerEntryType	Not null. What kind of movement this is.
`amount`	long	Not null, signed: positive adds points, negative removes them. Never 0.
`referenceId`	String (100)	Not null. The caller's order or purchase ID, so the entry can be traced and reversed.
`campaignId`	UUID	Optional. The campaign that boosted this earn, if any.
`description`	String (255)	Optional human-readable note, required for ADJUSTMENT.

Indexes: (account_id, created_at) for history pages. No update or delete ever runs on this table.

`LedgerEntryType` (enum, used by LedgerEntry)

`EARN, REDEEM, EXPIRE, REVERSAL, ADJUSTMENT`

PointLot

Extends: AuditableEntity | **Milestone:** M3 | **Table:** `point_lots`

What it's for: Tracks what is left of each earn, so redemption can use the oldest points first and the expiry job knows exactly which points to expire. Keeps the ledger itself append-only.

Field	Type	Rules and meaning
`accountId`	UUID	Not null. Owning account.
`earnEntryId`	UUID	Not null, unique. The EARN ledger entry that created this lot.
`originalAmount`	long	Not null, > 0. Points earned in that entry.
`remainingAmount`	long	Not null, between 0 and originalAmount. Reduced by redemptions and expiry.
`expiresAt`	Instant	Not null. 12 months after the earn.

Index: (account_id, expires_at) WHERE remaining_amount > 0, used both for oldest-first redemption and for the expiry job.

auth module

How a person proves who they are. Depends on the user module; the user module never depends on auth.

UserCredential

Extends: AuditableEntity | **Milestone:** M2 | **Table:** `user_credentials`

What it's for: Password login. A user who signs in only with Google has no row here.

Field	Type	Rules and meaning
`userId`	UUID	Not null, unique.
`passwordHash`	String (100)	Not null. BCrypt or Argon2 hash; the plain password is never stored.

UserRole

Extends: BaseEntity | **Milestone:** M2 | **Table:** `user_roles`

What it's for: What a user is allowed to do. Kept separate from credentials so Google-only users still have roles.

Field	Type	Rules and meaning
`userId`	UUID	Not null.
`role`	Role	Not null. Unique together with userId.

`Role` (enum, used by UserRole)

`USER, ADMIN`

ExternalIdentity

Extends: BaseEntity | **Milestone:** M2 | **Table:** `external_identities`

What it's for: Links a user to a Google account (Apple or others later), so Sign in with Google maps to the right user.

Field	Type	Rules and meaning
`userId`	UUID	Not null.
`provider`	AuthProvider	Not null.
`providerSubject`	String (255)	Not null. Google's stable sub claim. Unique together with provider. Link by this, never by email.
`email`	String (320)	The provider's email when linked, for reference only.

Only trust the provider email if email_verified is true.

`AuthProvider` (enum, used by ExternalIdentity)

`GOOGLE`

RefreshToken

Extends: AuditableEntity | **Milestone:** M2 | **Table:** `refresh_tokens`

What it's for: Long-lived token used to get new short-lived access tokens without logging in again, and to log out by revoking it.

Field	Type	Rules and meaning
`userId`	UUID	Not null.
`tokenHash`	String (64)	Not null, unique. SHA-256 of the token; the raw token is never stored.
`expiresAt`	Instant	Not null.
`revokedAt`	Instant	Optional. Set on logout or rotation; a revoked token is rejected.

Extends AuditableEntity because revoking it is an update.

campaign module

Campaign

Extends: SoftDeletableEntity | **Milestone:** M3 | **Table:** `campaigns`

What it's for: A promotion that changes how many points a purchase earns. The params are read by the matching Strategy class.

Field	Type	Rules and meaning
`name`	String (100)	Not null.
`ruleType`	CampaignRuleType	Not null. Picks the Strategy class via the Factory.
`params`	JSONB	Not null. Rule settings, for example {"multiplier": 2} or {"category": "electronics", "bonus": 500}.
`startAt`	Instant	Not null.
`endAt`	Instant	Not null, must be after startAt (checked in the entity and by a database CHECK).
`active`	boolean	Not null. Lets an admin pause a campaign without deleting it.

Index: (active, start_at, end_at) for finding campaigns that apply right now.

`CampaignRuleType` (enum, used by Campaign)

`MULTIPLIER, FIXED_BONUS, CATEGORY_BONUS`

audit module

AuditLog

Extends: BaseEntity + @Immutable | **Milestone:** M2 | **Table:** `audit_logs`

What it's for: Permanent record of admin actions and anything that changes points manually: who did what, to what, and when.

Field	Type	Rules and meaning
`action`	String (100)	Not null, for example CAMPAIGN_CREATED or POINTS_ADJUSTED.
`targetType`	String (50)	Not null, for example Campaign or PointAccount.
`targetId`	UUID	Optional. The ID of the changed record.
`details`	JSONB	Optional. Old and new values, reason given by the admin.

The actor is the inherited createdBy field, so no separate actorId column. Never updated or deleted.

shared module

IdempotencyRecord

Extends: BaseEntity | **Milestone:** M2 | **Table:** `idempotency_records`

What it's for: Remembers the response to each earn or redeem request, so a client retry with the same Idempotency-Key gets the same answer instead of awarding or spending points twice.

Field	Type	Rules and meaning
`userId`	UUID	Not null.
`idempotencyKey`	String (100)	Not null. Unique together with userId.
`requestHash`	String (64)	Not null. SHA-256 of the request body; same key with a different body returns 409.
`responseStatus`	int	Not null. HTTP status of the first response.
`responseBody`	JSONB	The first response, returned again on retry.

Location: shared/idempotency/. Delete records older than 24 hours with a scheduled cleanup. Moves to Redis in M4, with this table kept as the backup.

OutboxEvent

Extends: AuditableEntity | **Milestone:** M5 | **Table:** `outbox_events`

What it's for: An event saved in the same transaction as the ledger write, then published to RabbitMQ by a relay. Guarantees no PointsEarned or PointsRedeemed event is lost if the app crashes.

Field	Type	Rules and meaning
`aggregateType`	String (50)	Not null, for example PointAccount.
`aggregateId`	UUID	Not null. The entity the event is about.
`eventType`	String (100)	Not null, for example PointsEarned.
`payload`	JSONB	Not null. The event data sent to the queue.
`status`	OutboxStatus	Not null. PENDING when written, SENT after publishing.
`sentAt`	Instant	Optional. When the relay published it.

Location: shared/outbox/. Index: (status, created_at) for the relay's polling query. Extends AuditableEntity because status changes.

`OutboxStatus` (enum, used by OutboxEvent)

`PENDING, SENT`

Not an entity: ShedLock table (M4)

ShedLock manages its own `shedlock` table so the points-expiry job runs on only one of the 3 replicas. Create it with a Flyway migration; no Java entity is needed.